PYTHON COLLABORATIVE PROJECT

Student Energy Planner

A rule-based Python command-line tool for organizing study tasks and suggesting a time-bounded study plan.



PROJECT TEAM

Omansh Aneja • S26CSEU2199

Vansh Gandhi • S26CSEU2207

Yash Aggarwal • S26CSEU2206

Aditya Rao • S26CSEU2265

Ayush Kumar • S26CSEU2239

Our Team

Team roles and actual contributions should be confirmed by the group before final submission.

TEAM MEMBER	STUDENT ID	CONTRIBUTION
Omansh Aneja	S26CSEU2199	Core Developer – Designed the project logic, developed the priority algorithm, implemented the scheduling system, integrated modules, and handled final testing.
Vansh Gandhi	S26CSEU2207	Input & Data Management – Responsible for task input structure, data organization, and JSON-based storage module.
Yash Aggarwal	S26CSEU2206	Output & Testing – Responsible for output presentation, test-case planning, and functionality review.
Aditya Rao	S26CSEU2265	Documentation & Research – Responsible for project documentation, technical research, and concept explanation.
Ayush Kumar	S26CSEU2239	Presentation & Review – Responsible for presentation structure, visual communication, and project review

Introduction

Local command-line application • JSON persistence • Rule-based scheduling



WHAT THE PLANNER DOES

"Energy" means student-entered focus ratings. The program does not measure physiology, sleep, attention, or biological energy. Its plan is a rule-based suggestion based on user estimates.

Student Energy Planner is a Python command-line application that helps a student keep study tasks together and create a suggested plan for a chosen study window. A student records task and deadline information, estimated effort, difficulty, and focus need. The planner uses these details with current focus and available time to order tasks and propose study blocks.

Problem Statement

1

Many tasks

Assignments, revision, and other study work arrive together.

2

Different urgency

Some deadlines are closer than others.

3

Limited time

Available study hours must be shared across tasks.

PROBLEM TO SOLVE

How can a student organize tasks and create a clear, time-bounded work sequence using task urgency, estimated effort, focus needs, and available study time?

Students may have several assignments, revision tasks, and other study work competing for a limited number of hours.

Objectives

OBJECTIVE	INTENDED RESULT
Collect task details	Store task name, days remaining, deadline time, difficulty, estimated hours, and focus need.
Keep task data locally	Load and save task records through JSON.
Support prioritization	Order tasks with urgency first, then focus match, then difficulty.
Plan within constraints	Fit study blocks into both the bedtime window and maximum focused-study allowance.
Explain unfinished work	Show task minutes that remain unscheduled.

Features and Modules

MODULE	RESPONSIBILITY
Menu loop	Routes choices to add, view, plan, delete, or exit.
Task input and validation	Checks time fields, numeric ranges, positive study hours, and focus ratings.
JSON persistence	load_tasks() reads tasks.json; save_tasks() writes the current task list.
Task display / deletion	Shows numbered records and removes a selected record after confirmation.
Time helpers	Converts HH:MM to minutes and back, including midnight wrap.
Priority helper	Builds a tuple score from urgency, focus match, and difficulty.
Study planner	Creates study and break intervals, updates limits, and records leftovers.

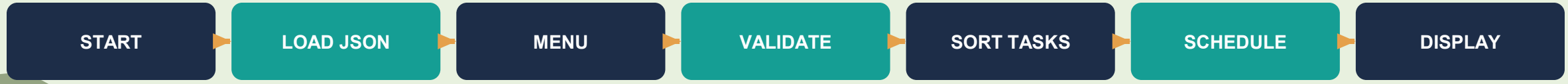
Algorithm and Flowchart

PLANNING STEPS

- Load saved tasks and show the menu.
- Validate current time, focus, bedtime, and maximum study hours.
- Convert times to minutes; move bedtime to the next day when it is earlier than or equal to start time.
- Sort tasks by the priority tuple.
- Add bounded study blocks and allowed breaks.
- Print scheduled intervals and any remaining work.

SCHEDULING RULES

- Each study block is the minimum of session length, task time left, bedtime-window time left, and focused-study time left.
- Focus 1-2: 20 minutes for first two sessions, then 25. Focus 3: 30 minutes. Focus 4-5: 40 minutes.
- Default break: 15 minutes. After study session 4 or 8, break length becomes 60 minutes.
- A break is added only after a full study block, when more work remains, enough clock time remains, and study allowance remains.
- Breaks consume clock time, not focused-study allowance.



Python Concepts Used

CONCEPT	PROJECT USE	WHY IT MATTERS
Strings, integers, floats	Task names, days, ratings, decimal work hours	Represent different input types
Lists and dictionaries	A list holds task dictionaries with named fields	Store structured task records
Functions	Separate task, time, priority, and planning actions	Organize reusable logic
Conditions and loops	Validate inputs; repeat menu and schedule steps	Choose paths and iterate
try / except	Handle invalid conversion and JSON/file errors	Recover from expected problems
JSON and files	json.load reads; json.dump writes tasks.json	Keep tasks between runs
Tuples and sorted()	Order (urgency, focus match, difficulty)	Create predictable priority
//, %, f-strings	Convert minutes, wrap hours, print times	Handle time and readable output

Computational Feature

Illustrative example: due-today task, 2 estimated hours, difficulty 5, focus need 4; start 09:00, focus 4, bedtime 14:00, cap 2 hours. Score = (5,5,5); available clock = 300 minutes; focused-study cap = 120 minutes.

PRIORITY SCORE

Score = (urgency, focus match, difficulty)

Focus match = 5 - abs(task focus need - current focus). Tuples compare left to right.

DAYS LEFT	POINTS
0-1	5
2-3	4
4-7	3
More than 7	2

STUDY SESSION LENGTHS

CURRENT FOCUS	STUDY BLOCK
1-2: first two sessions	20 minutes
1-2: later sessions	25 minutes
3	30 minutes
4-5	40 minutes

A block is further bounded by remaining task time and both schedule limits. Example values are illustrative, not a recorded run.

Testing and Results

ID	TEST SCENARIO	EXPECTED BEHAVIOR	OBSERVED RESULT
T1	First run; tasks.json missing	Begin with an empty task list	Program started successfully with an empty task list.
T2	Add valid task, restart	Saved task is loaded again	Task was saved successfully and loaded again after restarting the program
T3	Invalid time / rating	Reject value and prompt again	Invalid input was rejected and the program prompted the user to enter a valid value again
T4	Delete; decline confirmation	Keep selected task unchanged	Task remained unchanged after declining the deletion confirmation
T5	Start 22:00; bedtime 06:00	Treat bedtime as next day	Bedtime was correctly treated as the next day
T6	Task exceeds available hours	Stop at caps; report remainder	Study scheduling stopped at the available time limits and displayed the remaining task duration
T7	Plan with no tasks	Show message and return safely	Program displayed “Add some task first.” and returned safely

Screenshots

03 Task list

Show the saved records

```
4. FIRST TASK
   Deadline: in 0 day(s), at 18:30
   Difficulty: 4/5
   Study hours: 24.0
   Focus needed: 5/5
```

01 Main menu

Show available menu actions

```
PS C:\Users\omans\Desktop\PYTHEN COLLABORATIVE PROJECT> python -u "c:\Users\omans\Desktop\PYTHEN COLLA
BORATIVE PROJECT\main.py"

STUDENT ENERGY PLANNER
1. Add a task
2. View tasks
3. Make a study plan
4. Delete a task
5. Exit
Choose 1, 2, 3, 4 or 5:
```

02 Add task

Show validation and save confirmation

```
Choose 1, 2, 3, 4 or 5: 1
Task name: FIRST TASK
Deadline time (HH:MM, 24-hour format): 24:00
Enter a valid time, for example 18:30.
Deadline time (HH:MM, 24-hour format): 18:30
Days until the deadline? Enter 0 if due today: 0
Difficulty (1 to 5): 4
Estimated study hours: 24
Focus needed (1 = low, 5 = high): 5
Task added!
```

04 Study plan

Show blocks, breaks, and remainder

```
Choose 1, 2, 3, 4 or 5: 3
Enter current time (HH:MM, 24-hour format): 23:49
Your current focus level (1 = low, 5 = high): 4
Planned bedtime (HH:MM, 24-hour format): 06:30
Maximum focused study hours before bedtime: 3

Your current focus level: 4/5

TONIGHT'S SUGGESTED SCHEDULE
23:49-00:29 | Study | PHYSICS LAB (due today)
00:29-00:44 | Break | Take a short break
00:44-01:24 | Study | PHYSICS LAB (due today)
01:24-01:39 | Break | Take a short break
01:39-01:49 | Study | PHYSICS LAB (due today)
01:49-02:29 | Study | FIRST TASK  (due today)
02:29-03:29 | Break | Take a long break
03:29-04:09 | Study | FIRST TASK  (due today)
04:09-04:24 | Break | Take a short break
04:24-04:34 | Study | FIRST TASK  (due today)

DISCLAIMER: Some study work will remain after midnight.
- FIRST TASK : 22.5 hour(s) left; deadline in 0 day(s)
- DIGITAL DESIGN LEARN: 5.0 hour(s) left; deadline in 5 day(s)
- EVS GROUP PROJECT: 0.3 hour(s) left; deadline in 5 day(s)
Consider starting earlier or adjusting your workload.
```

Challenges and Learning

CHALLENGE	HOW THE PROGRAM HANDLES IT	LEARNING
Invalid input	Validation loops and specific exceptions	Check values before calculations
Task persistence	List of dictionaries saved as JSON	Separate data structure from file format
Clock arithmetic	Convert HH:MM into total minutes	Use one unit for interval math
Overnight plan	Add 1,440 minutes when bedtime crosses midnight	Represent next-day windows explicitly
Multiple time caps	min() bounds every study block	Update counters after each block
Priority ordering	Sort tuples left to right	Know how tie-breaks affect order
Source maintenance	Some session/break constants are declared but the loop uses local values	Keep configuration and active rules aligned

The code makes the following engineering considerations visible; add personal team examples if available.

Limitations

CURRENT IMPLEMENTATION

- Exact deadline time is stored and displayed but does not control task ordering or placement; days_left sets urgency.
- Focus and task effort are self-reported; the program does not measure attention, fatigue, or sleep.
- Final status messages use fixed “midnight” wording, even when the selected bedtime is different.
- There is no completion flag, so completed tasks can be planned again unless removed.
- Task data is stored locally in tasks.json; no cloud sync, calendar connection, or real-time integration is implemented.
- Missing or invalid JSON loads as an empty list; no backup or recovery history is documented.
- The planner produces suggestions and cannot guarantee task completion.

INTERPRETATION

The student should review and adjust the suggested schedule.

This is a local command-line application with deterministic rules. It does not use AI/ML, biological energy sensing, cloud services, or live calendar data.

Conclusion

CONCLUSION

Student Energy Planner demonstrates how introductory Python can combine input validation, JSON persistence, task prioritization, time arithmetic, and a bounded scheduling loop. The result is a transparent suggestion that the student can review.